

照章办事

“嗨，Tim，我是Jackie。我在使用化学品跟踪系统申请化学品时遇到了一个问题。实验室管理员建议我来问问你。他说你曾作为产品代言人给系统提供过很多需求。”

“是的，没错，”Tim回答道，“你碰到什么问题了？”

“我需要更多的光气来合成为我研究项目做的染料，”Jackie说，“但是系统拒绝了我的申请。它说我已经有一年多没有参加过处理危险化学品的培训了。这是怎么回事？我使用光气已经有好几年了，以前都没有碰到其任何问题。为什么这次不能申请了呢？”

“你可能知道Contoso制药公司要求员工每年都参加一个安全使用危险化学品的进修班，”Tim指出，“这是一个基于OSHA法规的公司政策。化学品跟踪系统在这方面做了强制。我知道以前无论你要什么，库房人员都会给你，不过现在不行了。很抱歉给你造成不便，不过要想让系统批准你使用更多的光气，必须参加进修班。”

每个组织的运营都要遵守很多政策、法规以及行业标准。银行、航空以及医疗器械制造这样的行业必须遵守大量政策法规。这样的控制原则统称为业务规则或业务逻辑。业务规则通常通过人工的政策和流程来保证。然而在许多情况下，也需要软件应用强化这些规则。

大多数业务规则起源于任何特定的软件应用之外。即使所有的化学品采购和分配是手动完成的，公司政策也要求员工每年都要参加处理危险化学品的培训。标准的会计实践在发明数字计算机之前已经使用很长时间了。由于业务规则是业务的属性，所以它们本身并不是软件需求。然而，业务规则是需求的来源，因为它们决定着系统必须具备哪些属性才能符合规范。如第1章中图1-1所示，业务规则可能是多种需求类型的来源。表9-1说明了业务规则是如何影响多种需求类型的，并提供了示例。

人们有时会把业务规则与业务过程或业务需求混为一谈。就像第5章所说，业务需求描述了组织期望的产出或概要目标，指出要构建或采购一个软件。业务需求是项目实施的理由。业务过程描述了将输入变为特定输出以达成特定结果的一系列活动。信息系统通常都业务过程自动化，因而有效率及其他好处（可以满足提出的业务需求）。业务规则通过建立词汇表、施加限制、触发行动或监控运算过程等方式影响业务过程。同一个业务规则可能应用到多个手动

或自动流程中，因此最好把业务规则当作一组独立的信息。

表 9-1 业务规则如何影响各类软件需求

需求类型	业务规则的影响	示例
业务需求	政府法规可能成为项目的某个业务目标	化学品跟踪系统必须符合近五个月内所有联邦和州的化学品使用与处理报告法规
用户需求	隐私策略决定哪些用户可以或者被禁止使用系统完成某些任务	只有实验室管理员可以为自己之外的其他人生成化学品接触报告
功能需求	公司政策规定所有供应商在收到发票之前必须经过核准登记	如果收到来自一个未注册供应商的发票，供货系统应该以电子邮件方式向供应商发送供应商登记表以及 W-9 表格的可编辑 PDF 文档
质量属性	来自政府机构(如 OSHA 和 EPA)的法规可以指明安全性要求，它们必须由系统功能强制执行	系统必须维护安全培训记录，在员工用户申请有害化学品前，必须保证他们接受过适当的培训

并不是所有公司都会将其核心业务规则当作珍贵的企业资产。有一些部门可能会以文档形式记录他们自己的规则，但是很多公司都不会统一安排人力记下业务规则并放在共享知识库中供整个组织查看。将如此重要的信息作为企业的非官方消息会导致各种问题。如果业务规则不加以正确记录和管理，就会局限于少数人的大脑中。业务分析师需要知道向谁打听影响他项目的规则。不同的人对规则可能有相互冲突的理解，导致不同的软件应用采取不同的方式实现同一个规则或是干脆完全忽略这个规则。拥有一个业务规则主知识库可以使受某些特定规则影响的所有项目都能找到规则并用一致的风格去实现规则。



陷阱 如果一些业务规则只有特定专家才知道，但没有文档记录，那么这些专家离开组织后就会产生知识真空区。

作为一个例子，你的组织很可能有几个安全策略来控制对信息系统的访问。这些策略可能会规定密码的最小和最大长度限制、密码更新频率、多少次登录错误后需要锁定用户账户等。组织开发的应用应当以一致的方式遵循这些策略——也就是业务规则。跟踪每个规则的实现代码，在规则有变时更新系统更容易了，例如修改密码更新周期。这种做法同样有利于在项目之间重用代码。

业务规则分类法

业务规则组织（2012）从业务及其信息系统两方面提出了业务规则定义。

- **从业务角度** “业务规则是一个指导原则，它描述了在某个特定活动或环境中的行为、行动、实践或流程所应尽的义务。”（规则应该有

一个明确的动机，并带有强制手段，而且必须指明违反规则会带来什么后果。)

- **从信息系统角度** “业务规则是对业务某些方面的定义或限制的声明。它的目的是维护业务结构或控制以及影响业务行为。”

基于业务规则的发现和文档化过程及其在自动业务规则系统的实现 (von Halle 2002; Ross 1997; Ross and Lam 2011) 发展出一整套方法论。除非是在构建一个严格的规则驱动系统，否则不需要如此复杂的方法论。只需要简单识别和记录与系统有关的规则，并将它们与实现它们的特定需求联系起来。

有很多分类方式可以用来组织业务规则 (Ross 2001; Morgan 2002; von Halle 2002; von Halle and Goldberg 2010)。简单的分类法如图 9-1 所示，包含 5 类规则，适用于大多数情况。第六类是对业务很重要的术语、专有词汇、短语以及缩写。可以按照实际业务规则来组织术语。词汇表是定义术语的另一个好地方。



图 9-1 一个简单的业务规则分类法

用一致的方式记录业务规则比热烈讨论如何精确划分规则重要得多。然而，一个分类法有助于识别出可能被忽视的业务规则。将规则分类有助于你思考如何将它们应用到软件应用中。例如，约束通常要求系统在功能上强制这些约束，提出功能，要求在某些条件下执行某些操作。让我们来看这五类业务规则的一些例子。

事实

事实就是业务在某个特定时间点简单而正确的陈述。事实描述重要业务术语之间的关联或关系。系统中重要数据实体的事实很有可能出现在数据模型中 (要想进一步了解数据建模，请参见第 13 章)。事实的例子如下所示：

- 每一个化学容器都有一个唯一的条形码标识
- 每一个订单都包含运费
- 销售税计算不包含运费
- 假如机票不可退，购买人更改行程会产生费用
- 高于 16 英寸的书摆放在图书馆的超大图书区

当然，业务总是有数不清的事实。收集无关事实会使业务分析停滞不前。即使它们是事实，也看不太清楚对开发团队有多大用处。将重点放在项目范围内的事实上，不要试图收集完所有业务知识集。尝试将事实联系到内外关系图

的输入/输出、系统事件、已知数据对象或者特定的用户需求。

约束

约束描述的是限制系统或其用户可执行的行为。人们可能这样描述一个约束类业务规则：某个行为必须、禁止或不得执行，或者只有某些人或者角色可以执行特定的行为。以下是来自不同来源的一些例子。

组织政策

- 未满 18 岁的贷款申请人必须有父母或法定监护人作为贷款担保人。
- 图书馆读者在同一时间内最多只能借 10 本书。
- 保险函中明文显示的投保人社会保险号码不得超过 4 位。

政府法规

- 所有软件应用必须符合适合视力障碍人士使用这一政府法规。
- 航空公司飞行员每 24 小时航程必须有连续 8 小时的休息时间。
- 除非有延期授权，否则个人联邦所得税申报表必须在 4 月 14 日后第一个工作日午夜之前寄出。

行业标准

- 抵押贷款申请人必须满足联邦住房管理局的资格标准。
- Web 应用不宜使用 HTML5 标准中废弃的任何 HTML 标签或属性。

太多约束

软件项目有很多类型的约束。项目经理必须在限定的日期、人力以及预算下开展工作。这种项目级别的约束属于项目管理计划。产品设计和实现约束代表施加的条件，如果不表述清楚，就只能依赖解决方案提供者的判断力了。此类开发人员的决策限制常见于 SRS 或设计规范。某些业务规则限制业务的运作方式，这些应当存储在业务规则库中。只要这些约束体现在软件需求中，就表明这些需求来源于哪个相关的规则。

业务规则约束即使不能直接映射到功能，也能够为软件开发传达一些隐性的信息。考虑一个零售商店的规定：只有主管和经理才能处理超出 50 美元的现金退款业务。如果是在开发一个 POS 机应用供商店员工使用，这条规则就意味着每个用户都要有一个权限级别。这个软件必须检查当前用户的权限级别是否能够执行某个操作，例如打开收银机抽屉，让收银员可以给用户退款。

因为很多约束类型的业务规则都涉及哪个类型的用户可以操作哪些功能，所以一种可以记录这些规则的简明方法就是使用角色与权限矩阵（Beatty and Chen 2012）。图 9-2 展示了一个公共图书馆信息系统中不同类型用户的矩阵。角色分为雇员与非雇员。系统功能分为系统操作、读者记录处理操作以及个

人的图书馆物品操作。表格中的 X 代表这一列的角色有权限执行这一行所示的操作。

角色与授权矩阵	员工	管理员	流动员工	图书馆助理	非员工	志愿者	读者
系统操作							
登录图书馆系统		X	X	X			
登记新的员工		X					
打印借书条		X	X	X			
读者记录							
查看读者记录		X	X				
编辑读者记录		X	X				
查看本人的记录		X	X	X		X	X
办理借阅卡		X	X				
接受罚款		X	X				
物品操作							
搜索馆藏目录		X	X	X		X	X
借出		X	X				
归还		X	X	X		X	
修改类别		X	X	X		X	
预约		X	X	X		X	X

图 9-2 约束性业务规则有时可以用角色及授权矩阵来表示

触发规则

当特定条件满足时触发某些活动的规则，称为触发规则。在手动过程中，可能需要一个人来完成这些活动。另一方面，这种规则也可能会提出软件功能方面的要求，使系统探测到触发事件时表现出正确的行为。触发行为的条件可能是某些独立真假条件的复杂组合。决策表（参见第 12 章）提供了一个简便的方式来记录涉及复杂逻辑的行为触发类业务规则。表格中“如果<某些条件为真或某些事件发生>，那么<某些事件会发生>”这样的语句很可能代表有人正在描述一个触发规则。以下是化学品跟踪系统中行为触发类业务规则的一些例子。

- 如果化学品库房有所需化学品容器的库存，就将库存容器发放给申请人。
- 在季度的最后一个日历日，按规定生成当前季度有关化学品处理的 OSHA 和 EPA 报告。
- 如果某个化学品容器有效期满，就需要通知容器当时的持有人。

业务有时会提出有利于商业性成功的一些规定。考虑一下网上书店如何利

用以下业务规则在客户购买某个特定产品后刺激进一步的冲动性购买。

- 如果客户购买的图书作者写过多本书，就在订单完成前向客户展示此作者其他著作的信息。
- 在客户将书放入购物车后，显示购买了此书的其他用户还购买了哪些相关书籍。



约束的负面效应

我最近在 Blue Yonder 航空公司用里程为我的妻子 Chris 兑换了一张机票。当我就要完成兑换时，BlueYonder.com 显示网站发生了错误，无法出票。它提示我立即电话联系航空公司。等我终于拨通预约代理的电话后，他告诉我航空公司无法通过邮件或者电子邮件发送里程兑换机票，因为 Chris 和我的姓氏不一样。我需要去航空公司售票处拿身份证领取兑换的机票。

此次事件是由一个约束性业务规则导致的，规则原文可能是这样的：“如果乘客和里程兑换人的姓氏不同，那么兑换人必须本人亲自领取机票。”这可能是为了预防欺诈。Blue Yonder 网站执行的这条规则降低了系统的可用性，并给顾客带来了麻烦。系统显示了一个错误警告信息，而不是直接告诉我存在姓氏不同的问题以及我应该怎么做。这浪费了我的时间、预约代理人的时间以及不必要的电话费。业务规则的草率实现可能会对顾客和业务造成负面影响。

推理

有时也称为推断的知识或派生的事实，推理通常是指从已知的事实中产生新的事实。与触发条件业务规则相同，推理也经常用“如果、那么”句型来描述，不过推理中的“那么”部分只是描述知识性的信息，而不是要执行的动作。推理的一些例子如下。

- 如果在 30 个日历日之内没有收到应付款项，这个账号就存在拖欠债务的问题。
- 如果供应商在收到订单的五天后仍然无法发货，就会被认为是延期交付的。
- 在小白鼠身上 LD50 毒性低于 5 mg/kg 的化学品是被认为有毒的。

运算

第五类业务规则定义了一些运算，通过使用特定的数学公式或算法将已知数据加工为新的数据。很多运算遵循的规则都是企业外部规则，例如所得税扣缴计算公式。以下是一些文字描述的运算类业务规则。

- 重量超过 2 磅的订单，国内陆路运费为 4.75 美元加上超出部分每盎

司 12 美分。

- 订单的总价为订单内物品单价的总和，去除折扣金额，加上发货地州、县的销售税，加上运费，再加上可选的保险费。
- 购买 6~10 个，单价享受 9 折优惠，购买 11~20 个享受 8 折优惠，购买 20 个以上享受 7 折优惠。

像这样用自然语言描述运算细节既冗长又难以理解。不过，可以用符号形式表达这些运算，例如提供数学表达式或者清晰且易于维护的规则表。表 9-2 就用更清晰的方式来描述前面的单价折扣运算规则。

表 9-2 使用表格描述运算类业务规则

ID	购买数量（个）	优惠比例
DISC-1	1~5	0
DISC-2	6~10	9 折
DISC-3	11~20	8 折
DISC-4	20 以上	7 折



陷阱 在记录业务规则集或是描述带范围的需求时，要小心边界值重叠问题。很容易将范围定义为 1~5，5~10 以及 10~20，此时 5 和 10 到底应该归于那个区间就会令人疑惑。

原子业务规则

假设你走向友好的图书管理员并问道：“DVD 我能借多久？”管理员回答说：“你可以把 DVD 或者蓝光碟片借走一周，并且在没有其他读者预约的情况下续借 2 次，每次 3 天。”管理员的回答基于图书馆的业务规则。然而她的回答是把多个规则糅合到一句话中。像这样组合业务规则会导致规则难以理解和维护。很难确认所有可能的条件都有全面覆盖到。如果好多个功能片段都追溯到这个复杂的规则中，那么将来规则中的一部分发生变化时就得花时间找到和修改相应的代码。

一个较好的策略是在原子级别记录业务规则，而不是在一条规则中组合多个细节。这可以使得规则简明扼要。还有利于规则的重用、修改以及任意组合。用原子方式记录推理知识以及触发条件业务规则，不要在“如果、那么”结构的前半段使用“或”逻辑，在后半段不要使用“与”逻辑（von Halle 2002）。可以将复杂的图书馆规则拆解为多个原子业务规则，如表 9-3 所示。（第 10 章介绍表 9-3 中所用的分层符号标记法）这些业务规则被称为原子的，因为它们无法被继续分解。最终可能会得到很多原子业务规则，而功能需求将依赖于这些规则的各种组合。

表 9-3 图书馆的一些原子规则

ID	规则
Video.Media.Types	DVD 碟片和蓝光碟片是视频物品
Video.Checkout.Duration	视频物品每次可以借走一周
Renewal.Video.Times	视频物品最多可以续借两次
Renewal.Video.Duration	续借视频物品可以将租借期延长 3 天
Renewal.HeldItem	被其他客户预约的物品不能被续借

为了说明使用原子业务规则是如何提高可维护性的，可以考虑这种情况：当下一代视频技术出现的时候，或者图书馆废弃 DVD 碟片时，只需要更新 Video.Media.Types 规则，其他规则则不受任何影响。

记录业务规则

因为业务规则可能影响很多应用，所以组织应当把自己的规则视为企业级资产进行管理。开始的时候，一个简单的业务规则目录就够了。如果使用需求管理工具，可以将业务规则作为一个需求类型进行存储，并保证所有软件项目都可以访问。大型组织或者其运营与信息系统均由业务规则驱动的组织应当建立业务规则数据库。如果规则目录超出文字处理软件、电子表格、维基或其他协同工具能处理的范围，最好使用一个商用的规则管理工具。一些业务规则管理系统包含规则引擎，可以在应用中自动化规则实现过程。业务规则组织（2012）维护着一个业务规则管理产品的列表。如果你在开发应用时发现了新的规则，将它们加入目录中而不是仅仅记录在这个特定应用的文档中，如果只是记录在源代码中，就更糟糕了。与安全、保密、财务或遵守法规相关的规则如果不加以妥善管理和强制执行，会带来极大的风险。



陷阱 不要让业务规则目录变得过于复杂。只要能保证开发团队有效使用，在可以用最简单的格式记录业务规则。规则库的所有者应该是业务部门，而不是 IT 部门或者项目团队。

对识别与记录业务规则有了更多的经验后，可以尝试用一些结构化的模板来定义不同类型的规则（Ross 1997；von Halle 2002）。这些模板描述了关键字模板和句式，力求用一致的风格结构化规则记录。它们同样有利于将规则存储到数据库、商用的业务规则管理工具或业务规则引擎中。相关规则的分类也可以用决策树、决策表（特别是涉及复杂的逻辑时）或角色与权限矩阵之类的工具来表达。不过一开始的时候可以尝试如表 9-4 所示的简单格式（Kulak and Guiney 2004）。

表 9-4 一些业务规则目录的例子

ID	规则定义	规则类型	静态或动态	来源
ORDER-5	如果顾客购买书的作者还写过其他书，那么在交易结束前向他展示作者的其他著作	触发规则	静态	××销售政策
ACCESS-8	网页上的所有图片都必须有文本的替代描述供电子阅读设备使用以满足视力不良用户的使用需求	约束	静态	无障碍设计的 ADA 标准
DISCOUNT-13	折扣基于当前交易的额度计算，算法见表 BR-060	运算	动态	公司的××定价政策

为每个业务规则提供一个唯一 ID 可以帮助将需求链回到特定规则，与之联系起来。例如，某些用例的模板包含一个字段用于记录影响这个用例的业务规则。不需要将完整的规则定义写入用例的描述，需要填入相关规则的 ID 即可。每个 ID 都可以指向业务规则实体。这样一来，就不用担心规则更改之后会造成用例失效。



隔离法则

航空交通管理系统(ATC)必须保证在至少四个维度(高度、经度、纬度和时间)对航空器进行区分，以避免事故发生。航空器机载系统、飞行员、地面控制器以及 ATC 系统自身需要根据上百个信息来源计算飞行轨迹和速度以便确认某飞机可能在何时与其他飞机距离太近。很多业务规则需要确保最小的法定相互距离和时间。这些规则是动态的，随着科技的发展(例如用 GPS 定位取代雷达)以及法规的变化，它们会周期性更新。这就意味着系统需要经常接受一组新的规则，验证规则自身的一致性与完备性，然后在飞行员和控制器使用新的规则后也切换至新规则。曾经有个 ATC 项目一开始就将这类业务规则的当前值硬编码到系统中，认为它们是不变的。当干系人认识到需要周期性更改这些安全规则时，却发现系统需要大量返工。

“规则类型”这列指出每个业务规则是事实、约束、触发、推理还是运算。“静态或动态”列说明规则将来变化的可能性。这个信息对开发人员很有帮助。如果他们知道特定的某些规则是周期性变化的，就可以通过改变软件架构来使受影响的功能或数据易于更新。所得税计算至少每年变更一次。如果开发人员将所得税信息放在表格或数据库中，而不是写死在系统代码中，更新这些值的时候就会十分方便。物理定律硬编码没有什么问题，例如基于热力学定律的计算；相比之下，人为的规则更容易。

表 9-4 中的最后一列说明了规则的来源。业务规则的来源包括企业以及管理规定；主题专家和其他人员；政府规章制度之类的文档。知道来源以后，人们就知道应该去哪里找关于规则的更多信息或者跟踪规则变化。

发现业务规则

只问“你的需求是什么”对引导用户需求并不会太多帮助，问用户“你们的业务规则是什么？”效果也不会好到哪里去。有时候在你投入一定时间后会发明一些业务规则，有时候，它们会在需求讨论中自动浮现，有时候你需要主动捕获它们。Barbara von Halle（2002）提出了一个用来发现业务规则的完整流程。以下是一些常见的规则发现方式和地点（Boyer and Mili 2011）。

- 组织内的“常识”，通常来自从事此业务很长时间的个人，他们通常都知道业务的运作细节。
- 其需求和代码中已内嵌业务规则的遗留系统。这时需要对需求或代码的原理进行反向工程来理解背后的规则。用这种方式得到的业务规则有时是不完备的。
- 业务过程建模，通常引导分析师找出可能影响每个流程步骤的规则：约束、触发事件、运算规则以及相关的事实。
- 现有文档分析，包括以往项目的需求规范说明、法规、行业标准、企业规范、合同以及业务计划书。
- 数据分析，例如数据对象可以有的各种状态以及在什么情况下用户或系统事件可以改变对象的状态。这种授权也可能由图 9-2 所示的类似角色和授权矩阵表示，它们能提供关于用户权限级别及安全性规则的信息。
- 公司其他部门构建合规系统。

仅仅因为从这些不同来源发现了一些业务规则，并不代表它们适用于你当前的项目，它们甚至有可能已经不再有效了。遗留应用代码里实现的运算规则可能已经过时。一定要确认从旧的文档或应用中收集到的规则是否需要更新。评估你发现的规则之适用范围。它们是只适用于某项目，还是涉及一个业务域或整个企业？

通常，项目干系人早就知道哪些业务规则会对应用产生影响。有些员工有时会处理某些特定类型的规则。如果你遇到这种情况，就找到他们并且邀请他们加入讨论。业务分析师在产出其他需求产物以及模型引导活动中也可以收集业务规则。在当面沟通以及工作坊上，业务分析师也可以通过提问的方式来琢磨用户所提出的需求和约束背后的原理。这种讨论通常会使业务规则作为基本原理浮现出来。图 9-3 展示了一些潜在的规则来源。它也提供了一些问题供业务分析师用于与用户讨论各种需求。

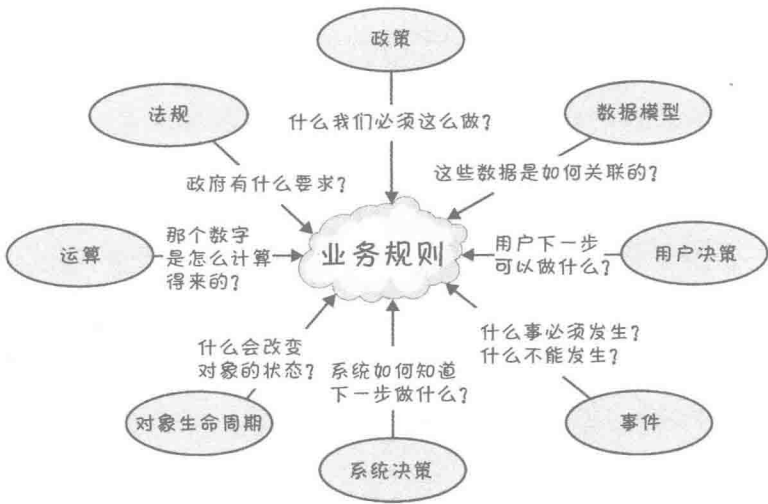


图 9-3 从不同角度提问，发现业务规则

业务规则与需求

识别并记录业务规则之后，需要决定哪些必须在软件中实现。业务规则及其对应的功能需求有时候看起来很像。然而，规则是来自外部的策略声明，必须通过软件强制执行，因此提出了系统功能要求。每个业务分析师都必须决定哪些规则和自己的应用有关，哪些必须由系统强制执行，如何强制执行。

回想一下化学品跟踪系统的约束规则，在用户申请危险化学品的时候，系统会检查此用户的培训记录是否过期。为了遵守这条规则，根据化学品跟踪系统是否可以访问培训记录数据库，分析师可能会得出不同的功能需求。如果可以访问，系统可以查找用户的培训记录并且决定是接受还是驳回申请。如果记录无法在线获取，系统也许需要暂时存储化学品申请，并向培训联系人发消息，由他批准或拒绝请求。在以上两种情况下，规则并没有什么不同，但是软件功能（当执行过程中遇到业务规则时需要采取什么行动）随着系统所处环境的不同会有所不同。

再举一个例子，考虑以下规则。

- 规则#1（触发规则）：“如果化学品容器临近有效期，需要通知当时的容器持有人。”
- 规则#2（事实）：“容器内所装化学品会分解为易爆物，那么容器的保质期为一年。”

规则#1 是“向化学品持有人提示过期时间”这个系统特性的来源。再加上类似规则#2 的其他规则，可以帮助系统明确哪些容器存在过期时间，因而需要在合适的时间提醒其的持有人。例如，一罐开封的乙醚会变得很危险，因为接触氧气会使它产生易爆的副产品。基于这个规则，化学品跟踪系统显然必

须监控有保质期的化学品容器的状态，并且通知正确的人归还容器以便安全地销毁。业务分析师需要为这个特性推导出一些功能需求，例如以下几条。

- **Expired.Notify.Before:** 如果一个有保质期的化学品容器的状态是未被销毁，那么系统需要在容器过期前一周通知容器当时的持有人。
- **Expired.Notify.Date:** 如果一个有保质期的化学品容器的状态是未被销毁，那么系统需要在容器过期当天通知容器当时的持有人。
- **Expired.Notify.After:** 如果一个有保质期的化学品容器的状态是未被销毁，那么系统需要在容器过期一周后通知容器当时的持有人。
- **Expired.Notify.Manage:** 如果一个有保质期的化学品容器的状态是未被销毁，那么系统需要在容器过期两周后通知容器当时持有人的经理。

当你遇到一组像上面这样非常相似的规则时，考虑把它们记录在一个表格里而不是罗列出来（Wiegiers 2006）。那样更简洁并且易于检查、理解与修改。同时也可以更简洁的标记需求，因为表中只需显示追加到父需求标记后的后缀即可。下面是上述 4 个功能需求的另一种表现形式。

- **Expired.Notify** 如果一个有保质期的化学品容器的状态是未被销毁，那么系统需要在下表所示的时间通知表中相应的联系人。

需求标识	被提醒人	提醒时间
.Before	容器当前的持有人	过期日一周之前
.Date	容器当前的持有人	过期当天
.After	容器当前的持有人	过期日一周之后
.Manager	容器当前持有人的经理	过期日两周之后

把一切串起来

为了避免冗余，不要将业务规则目录中的规则复制到需求文档中。在特定功能与算法的来源描述中引用相应的规则。有好几种方法可以用来定义功能需求及其父业务规则的联系，下面是其中三种。

- 如果使用需求管理工具，就为需求新建一个“来源”属性并把原始规则列为功能需求的来源。（参见第 27 章）
- 在需求跟踪矩阵或需求映射矩阵中定义功能需求与相应业务规则之间的可跟踪链接（Beatty and Chen 2012）。当业务规则与需求保存在同一个库中时，使用这种方法最方便。（参见第 29 章）
- 如果业务规则和需求存储在文字处理软件或电子表格文件中，在需求描述中插入业务规则 ID 的超链接并将其指向记录业务规则描述的文件。注意，如果规则集的存放地址出现变化，超链接很容易被损坏。这些链接可以使需求始终能引用最新版本的规则，因为需求中只是简单记

录了指向规则实体的一个指针。如果规则更新了，搜索规则 ID 就能找到可能需要做相应修改的需求或已实现的功能。像这样使用链接有助于在多个地方与项目中重用相同的规则，因为规则不会埋没在任何特定应用的文档中。不过，阅读 SRS 的开发人员需要跟随交叉引用的链接才能找到规则细节。这是你选择不复制信息的代价（Wiegiers 2006）。

由于需求工程包罗万象，所以管理业务需求不存在普遍适用于所有情况的简单而完美的解决方案。但是当你开始积极寻找，记录并应用业务规则之后，应用程序开发决策背后的原因对所有干系人就会变得更加清晰可见了。



行动练习

- 尝试在当前项目中为表 9-1 中的每种业务规则类型分别找到至少一个对应的业务规则。
- 为当前项目涉及的规则整理出一个业务规则目录。根据表 9-1 的提示将规则进行分类，并记录每个规则的来源。
- 建立一个可追踪矩阵记录你识别出的业务规则分别由哪个功能需求来保证执行的。
- 根据每个功能需求背后的原理，发现其他隐含的业务规则。